Informe individual - Gestión de vulnerabilidades

Grupo: 04 · **Autor:** Agustin Nicolas Ferrari
GitHub: FerrariAgustinNicolas · **Destinatario:** Gerencia de Operaciones
Fecha de revisión: 01/11/2026

Decisión solicitada: aprobar responsables, contención proporcional ante señales corroboradas y una ventana urgente para portal, API y Python. Recuperar disponibilidad no basta para cerrar un incidente: también debe evaluarse el riesgo residual.

1. Situación y nivel de riesgo

La consigna pide una organización ficticia asignada, pero no indica cuál corresponde al grupo 04 [1]. Mantengo como supuesto una empresa con portal PHP de legajos, API Node.js y servicio Python interno; no lo presento como asignación docente. Mi aporte es documental; las pruebas del taller las realizó Lorenzo Bellomo [2].

PHP2Shell permite ejecutar comandos sin autenticación; Node2Shell afecta al componente que descomprime Brotli. El contexto actualizado de la cátedra informa explotación y KEV para ambas familias [4]. El riesgo es alto: priorizo el portal por datos personales, sin posponer la API que recibe contenido externo ni Python con señales sospechosas en el escenario. La exposición pública del portal y API es un supuesto. Inventariaría también proxies y dependencias: la gravedad numérica no basta.

Hay dos problemas distintos: vulnerabilidad e incidente. Las capturas muestran actualizaciones; el log sintético representa procesos anómalos, no una intrusión en los contenedores del grupo. Lo trataría como incidente presunto dentro del escenario, sin afirmar exfiltración ni atribuir los eventos a un CVE concreto. Parchear reduce exposición, pero no expulsa a un atacante que ya hubiera ingresado.

2. Plan priorizado con plazos, responsables y reversión

Los plazos cuentan desde el aviso y son propuestas, no acciones realizadas por mí. Operaciones coordina; seguridad valida las decisiones de contención y recuperación.

1. **Primera hora - Seguridad y operaciones:** corroborar alertas, preservar registros con origen, hora e integridad y aislar selectivamente los activos sospechosos. Revertir el aislamiento solo con aprobación y evidencia de recuperación; no por ausencia de nuevas alertas.
2. **Primera hora - Dueños del portal y API:** mantener la mitigación PHP y evaluar rechazo de Brotli antes del componente que lo descomprime. Verificar configuración efectiva e impacto funcional. Retirar una compensación únicamente al resolver su riesgo, con validación del servicio.
3. **Menos de 24 horas - Dueño del portal:** actualizar desde una imagen confiable; comprobar versión, inicio de sesión, consulta y carga de legajos. Si falla una operación crítica, suspender o recuperar una imagen limpia aislada; no reabrir al tráfico una versión vulnerable para recuperar disponibilidad.
4. **Menos de 24 horas - Dueños de API y Python:** actualizar en emergencia por el contexto de explotación y probar solicitudes habituales y Brotli benigno. La contención de actividad corroborada es inmediata, sin esperar al parche.
5. **Antes de reconectar - Seguridad y operaciones:** investigar persistencia, reconstruir desde imágenes confiables y revocar credenciales expuestas desde un entorno seguro. No restaurar secretos comprometidos ni una imagen sospechosa.
6. **Primera jornada - Operaciones, comunicación y legales:** informar hechos, incertidumbres y medidas comprobadas; fijar la próxima actualización. Rectificar el mensaje si cambia el alcance, sin afirmar contención o acceso a datos no probados.

Alcance pendiente del host: la cátedra incorpora la cadena PHP2Shell + Fragnesia (CVE-2026-46300) [4]. Infraestructura debe relevar, durante la primera jornada, el kernel Linux del entorno Docker, incluida su VM si corresponde, y las correcciones del proveedor. No se evaluó su aplicabilidad: no afirmo que esté afectado. Parchear PHP no corrige el kernel; seguridad debe evaluar ese riesgo antes del cierre.

3. Indicadores de compromiso y criterios de descarte

El archivo del ejercicio registra Apache creando un shell con id y un proceso wget, Node creando un shell con curl, y Python creando uno con uname [2]. Son señales prioritarias porque no corresponden al comportamiento esperado de las aplicaciones; el resultado de las descargas no está registrado.

Para confirmar ejecución no autorizada, correlacionaría procesos con peticiones, usuario efectivo, archivos creados, conexiones y accesos a datos. Una acción sin autorización, respaldada por registros íntegros y contexto del servicio, permitiría confirmar ese hecho; no demostraría automáticamente exfiltración o el vector inicial.

Para descartar una sospecha concreta, exigiría una explicación verificable. Por ejemplo, una sesión SSH de db01 debe coincidir con usuario, origen y tarea administrativa autorizada. Eso puede explicar esa sesión, pero no descarta los eventos de los demás hosts. Concluir ausencia de compromiso requiere cobertura y búsqueda suficientes, no solo ausencia de coincidencias.

Límite actual: hay nueve minutos de log sin UID, resultados de comandos ni telemetría de red. Una descarga fallida, un parche instalado o un servicio saludable no descartan ejecución previa, lectura de secretos ni actividad en memoria. Sin más evidencia, mantendría el alcance como indeterminado y ampliaría la investigación antes de aprobar el cierre.

4. Transferencia a CVE-2026-115560

El aviso PHP del 23/10 describe condiciones específicas: progreso de subida activo y cambio del serializador de sesión de php_binary o php_serialize a php durante la ejecución, antes de iniciar la sesión. Recomienda desactivar el progreso de subida. No corresponde asumir que toda aplicación PHP está afectada ni que la actualización del 21/10 resuelve este caso [3].

Aplicaría el mismo marco, pero comprobaría primero esas condiciones en configuración y código. El dueño del portal realizaría esa revisión durante la primera jornada; mientras tanto conservaría la mitigación y registraría la incertidumbre. Si existe exposición, coordinaría una solución respaldada por el proveedor y pruebas de sesiones y cargas. Si no se cumplen las condiciones, documentaría la evidencia y mantendría seguimiento. No inventaría una versión corregida ni una verificación que no ejecuté.

5. Mejoras de proceso para la próxima vez

Mantendría inventario con exposición, dependencias y dueños; imágenes confiables; cambios con pruebas y reversión; y registros con identidad de host, usuario, procesos y resultados. Seguridad debe definir retención y cobertura según las necesidades de investigación. Cada simulacro cerraría con una revisión de evidencia faltante.

Mediría activos inventariados, tiempo hasta la contención, actualizaciones verificadas y pruebas funcionales completadas. Operaciones revisaría estos resultados después del ejercicio para corregir demoras. Mi conclusión es separar observación, inferencia y decisión: comunicar incertidumbre con precisión es más útil que declarar seguridad sin pruebas.

Fuentes

[1] Cátedra: consigna publicada y rúbrica individual.

[2] Grupo 04: entrega y capturas y log sintético, revisión 0ba3296.

[3] PHP: aviso informativo del 23/10/2026, sobre progreso de subida. Fuentes consultadas el 01/11/2026.

[4] Cátedra: contexto actualizado, Node2Shell y Fragnesia, revisión 23cbf681; consultado el 01/11/2026.